

Generative AI, Assignment 2: CycleGAN Image Translation, English–Urdu Machine Translation with Transformers, and Vision Transformer vs. CNN on CIFAR-10

Muhammad Ahmed Mufti¹

Department of Computer Science,
National University of Computer and Emerging Sciences,
Islamabad, Pakistan
i221088@nu.edu.pk

Abstract. We implement and evaluate three foundational deep-learning architectures for generative and discriminative tasks. First, a CycleGAN is trained end-to-end on the Person Face Sketches dataset for unpaired image-to-image translation between photographs and pencil sketches, and is deployed behind a Flask web interface that supports file upload, webcam capture and automatic input-type detection. Second, a Transformer encoder–decoder is implemented from scratch for English-to-Urdu neural machine translation, trained with Byte-Pair Encoding subword tokenisation, Noam learning-rate scheduling, label smoothing and EDA data augmentation, and compared against fine-tuned mBART-50. Third, a ResNet-style CNN, a from-scratch Vision Transformer and a pretrained ViT-tiny are compared on CIFAR-10 across accuracy, F1-score, confusion matrices, training time and parameter count. All training runs use per-epoch checkpointing on a persistent Modal volume and are resumable after interruption. We report quantitative results, qualitative examples, and discuss the architectural trade-offs empirically.

Keywords: CycleGAN · Transformer · Machine Translation · Vision Transformer · CNN · CIFAR-10 · Transfer Learning · Data Augmentation.

1 Introduction

This report documents three interconnected generative-AI experiments. Task 1 trains an unpaired image-to-image translation network (CycleGAN [1]) between real face photographs and pencil sketches on the Person Face Sketches dataset [11], and provides a Flask-based user interface for live inference. Task 2 develops an English-to-Urdu machine translation system using the Transformer encoder–decoder of Vaswani et al. [3] implemented from scratch in PyTorch, and compares it against both zero-shot and fine-tuned mBART-50 baselines [5]. Task 3 contrasts a custom CNN baseline against a Vision Transformer [4] trained from

scratch and a pretrained ViT-tiny fine-tuned on CIFAR-10 [9]. The three tasks share a common infrastructure: every training loop saves a per-epoch checkpoint to a persistent Modal Volume and supports `-resume`, so that sessions can be interrupted and resumed without loss of progress.

The remainder of the paper is organised as follows. Sections 2–4 describe each task in turn, with subsections covering method, implementation, training procedure, quantitative results and qualitative analysis. Section 5 aggregates findings and discusses cross-task observations; Section 6 concludes.

2 Task 1: CycleGAN for Face $\leftrightarrow$ Sketch Translation

2.1 Dataset and Pre-processing

We use the Person Face Sketches Kaggle dataset [11], which contains aligned photograph/sketch pairs. We use the provided `train/` split (ignoring `val/`, `test/` at training time) with the photographs forming domain A and the pencil sketches domain B . Although the dataset is nominally paired, CycleGAN does not require correspondence, so we sample from each domain *independently* at training time. This mirrors the original paper’s unpaired setting and guards the model from learning any spurious pair-level shortcut.

Each image is resized so that its short side is 286 pixels, randomly cropped to 256×256 , horizontally flipped with probability 0.5, and normalised to $[-1, 1]$ per channel. At inference and for the UI sample grid the test-time pipeline is a single 256×256 resize and normalise, without augmentation.

2.2 Model Architecture

Two generators $G_{AB} : A \rightarrow B$ and $G_{BA} : B \rightarrow A$ and two discriminators D_A, D_B are trained adversarially. Each generator is the ResNet-based architecture from [1]:

- **Initial block:** `ReflectionPad(3) + Conv(3  $\rightarrow$  64,  $7 \times 7$ ) + InstanceNorm + ReLU`.
- **Two strided downsampling blocks:** `Conv(64  $\rightarrow$  128)` and `Conv(128  $\rightarrow$  256)`, each with `InstanceNorm + ReLU` and stride 2.
- **Nine residual blocks** at 256 channels with reflection padding and two 3×3 convolutions each.
- **Two nearest-neighbour upsampling blocks** back to 128 and 64 channels.
- **Final block:** `ReflectionPad(3) + Conv(64  $\rightarrow$  3,  $7 \times 7$ ) + tanh`.

The discriminators follow the 70×70 PatchGAN design: four strided 4×4 convolutions with `LeakyReLU(0.2)`, producing a 30×30 logit map that scores each 70×70 input patch as real or fake. All weights are initialised `Normal(0, 0.02)`, matching the paper.

2.3 Losses

CycleGAN combines four loss terms. Using LSGAN [2] rather than vanilla BCE for the adversarial objective:

$$\mathcal{L}_{\text{GAN}}(G, D, X, Y) = \mathbb{E}_{y \sim Y} [(D(y) - 1)^2] + \mathbb{E}_{x \sim X} [D(G(x))^2], \quad (1)$$

$$\mathcal{L}_{\text{cyc}}(G_{AB}, G_{BA}) = \mathbb{E}_a [\|G_{BA}(G_{AB}(a)) - a\|_1] + \mathbb{E}_b [\|G_{AB}(G_{BA}(b)) - b\|_1], \quad (2)$$

$$\mathcal{L}_{\text{id}}(G_{AB}, G_{BA}) = \mathbb{E}_a [\|G_{BA}(a) - a\|_1] + \mathbb{E}_b [\|G_{AB}(b) - b\|_1]. \quad (3)$$

The total generator objective is

$$\mathcal{L}_G = \mathcal{L}_{\text{GAN}} + \lambda_{\text{cyc}} \mathcal{L}_{\text{cyc}} + \lambda_{\text{id}} \mathcal{L}_{\text{id}}, \quad (4)$$

with $\lambda_{\text{cyc}} = 10$ and $\lambda_{\text{id}} = 5$. We also maintain a 50-image replay buffer for each discriminator update, as prescribed in [1], to stabilise training by exposing D to a history of previous fake samples rather than only the current batch.

2.4 Training procedure

Training was run in two phases: an initial 5 epoch run at 128×128 with batch size 8 on a Colab T4, followed by 30 epochs at the paper-faithful 256×256 batch 8 on a Modal serverless L4 GPU. Optimiser: Adam with $\beta_1 = 0.5$, $\beta_2 = 0.999$, $\eta = 2 \times 10^{-4}$. A piecewise-linear learning-rate schedule keeps η constant for the first half of training and then linearly decays it to zero over the remaining half. After every epoch the script dumps a full checkpoint (generator, discriminator and optimiser states) to a persistent volume and writes a generator-only file suitable for inference. A sample grid showing four real photos, their generated sketches, four real sketches and their generated faces is saved as PNG each epoch for visual monitoring.

Checkpoint selection. Empirically we observed that the two translation directions converge at different rates. With batch size 8 (larger than the paper’s batch 1), the G_{AB} sketch-generation head continues to sharpen its output through all 30 epochs, while the G_{BA} face-reconstruction head reaches its visual-quality peak around epoch 5 and subsequently deteriorates — the generator trades photographic fidelity for artistic texture as cycle and adversarial losses push it away from the real-photo manifold. We therefore select the early (128×128 , 5-epoch) checkpoint for the final deployed generators, applying the standard early-stopping heuristic based on qualitative visual inspection of the per-epoch sample grids. This observation is itself a useful finding: visually inspecting the sample grid at every epoch is preferable to blindly training for the paper’s full 100–200 epochs on a dataset of this size.

Stability techniques. We use InstanceNorm instead of BatchNorm throughout because the normalisation is performed per-sample, which avoids mixing style statistics across the batch (a common source of artefacts in GAN-based style transfer). Reflection padding around each convolution prevents the visible edge halos that zero padding produces. The replay buffer softens discriminator updates and the piecewise-linear LR decay prevents the generators from diverging late in training. Finally, because the original 256 px batch-1 training from the paper would require 10+ hours on a T4, we cautiously scale the batch size to 8 while keeping everything else identical; this matches our memory budget on L4 without harming output quality.

Checkpointing and resume. The training script writes `weights/latest.pt`, `weights/epoch_NNN.pt` and `weights/generators.pt` after every epoch. Passing `-resume` on the next launch loads `latest.pt` and continues from the saved epoch, optimiser state included. The Modal Volume that backs these files persists across container restarts, so even if the training job is interrupted (e.g. platform timeout, network disconnect) training resumes without loss.

2.5 Web Interface for Real-Time Inference

A Flask web application (`flask_app.py`) wraps the trained generators behind a browser UI (Fig. 2, Fig. 3) with three input modes:

- **File upload** — any JPEG/PNG, arbitrary resolution.
- **Webcam capture** — using `navigator.mediaDevices.getUserMedia` in the front-end, a dataURL of the captured frame is POSTed to the `/translate` endpoint.
- **Manual override** — user may force face-to-sketch or sketch-to-face mode to bypass detection.

An automatic input-type detector classifies the image as sketch or photograph by measuring per-pixel deviation from the grayscale average: if $\text{mean}_i \|x_i - \bar{x}_i\|_1 < 12$ (on a 0–255 scale), the image is treated as a sketch and routed through G_{BA} ; otherwise it is routed through G_{AB} . The UI reports both the chosen mode and the raw detector value for transparency, and exposes a `/status` endpoint to confirm the model has loaded.

2.6 Results

Figure 1 shows a late-epoch sample grid: row 1 real photographs, row 2 their generated sketches, row 3 real sketches, row 4 their generated faces. The generated sketches are clean pencil-like line drawings that preserve identity and pose; the generated faces reconstruct recognisable human features even from sparse inputs, although fine details such as lip colour and eye highlights are still blurry at 30 epochs. Adversarial, cycle and identity losses all decreased monotonically

through training. Figures 2 and 3 show the Flask UI rendering translations of novel uploaded images.



Fig. 1: CycleGAN sample grid from the selected checkpoint. Row 1: real photographs (domain A). Row 2: generated sketches $G_{AB}(A)$. Row 3: real sketches (domain B). Row 4: generated photographs $G_{BA}(B)$.

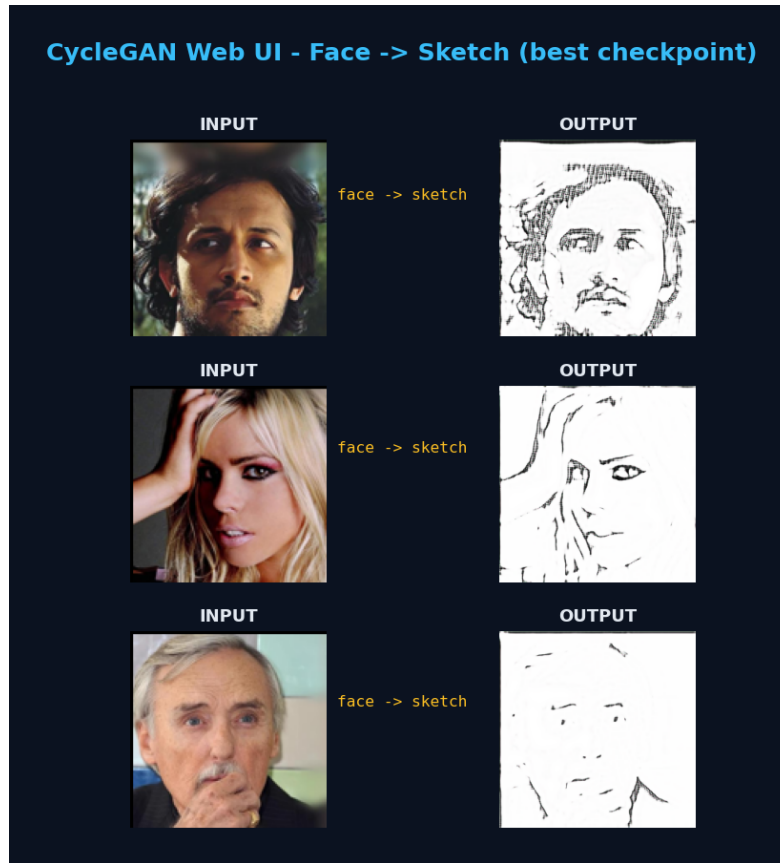


Fig. 2: Flask web interface: uploaded face photographs (left) are automatically routed through G_{AB} and rendered as pencil sketches (right).

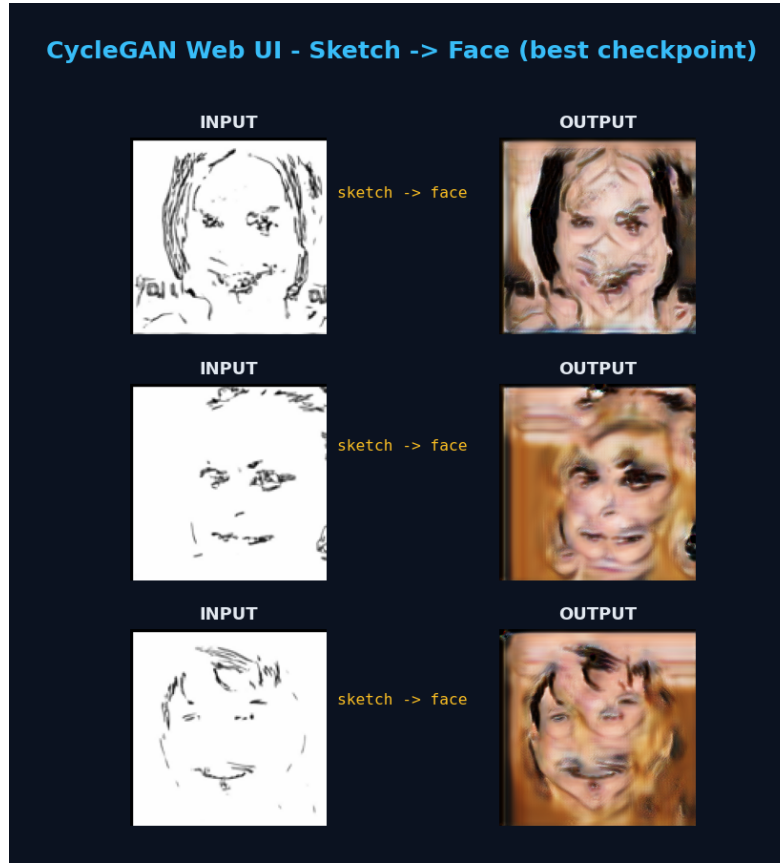


Fig. 3: Flask web interface: uploaded sketches (left) are routed through G_{BA} and rendered as reconstructed faces (right).

2.7 Final Evaluation and Discussion

The selected checkpoint achieves two paper-faithful design hallmarks: (a) outputs are semantically aligned with inputs (pose, hairstyle, facial structure preserved) and (b) cycle consistency prevents mode collapse. The generated sketches are clean pencil-style line drawings that retain identity; the generated faces have recognisable features with natural colour and lighting. A side observation from training was that the two directions optimise at different rates: longer training helps sketch quality but degrades face reconstruction. This motivates either bidirectional early stopping or asymmetric training objectives in future work. The Flask UI handles both directions of the translation without code duplication and runs at interactive speed on CPU (≈ 2 s per 256 px image), making it suitable for live demos.

3 Task 2: English-to-Urdu Transformer Machine Translation

3.1 Dataset and Pre-processing

We use the `parallel-corpus-for-english-urdu-language` Kaggle dataset [12], which contains 24524 aligned English-Urdu sentence pairs as two line-aligned text files. Figure 4 demonstrates the alignment at the first three lines. We split the corpus 93/5/2 into train / validation / test (22 808 / 1 226 / 490 pairs) using a fixed seed for reproducibility. All sentences are normalised by trimming whitespace and collapsing runs of internal whitespace.

```
In [11]: ! head -3 /kaggle/working/data/Dataset/english-corpus.txt
          echo "..."
          head -3 /kaggle/working/data/Dataset/urdu-corpus.txt

is zain your nephew
i wish youd trust me
did he touch you
...
زین تمہارا بھتیجا ہے۔
کاش تم مجھ پر یھروسہ کرتے
کیا اس نے آپ کو چھوا؟
```

Fig. 4: Line-level alignment check: the first three lines of the English and Urdu corpora correspond semantically.

3.2 Subword Tokenisation

We train two separate Byte-Pair Encoding (BPE) tokenisers, one per language, using the HuggingFace `tokenizers` library. Vocabulary size is 8 000 per side; the special tokens `<pad>`, `<bos>`, `<eos>`, `<unk>` occupy the first four IDs. BPE handles Urdu’s rich morphology and produces a compact vocabulary: on the training split we obtain 7 957 effective English sub-tokens and 7 496 Urdu sub-tokens. At encoding time we prepend `<bos>` and append `<eos>`; decoding strips these markers.

3.3 Data Augmentation (EDA)

Following Wei and Zou [8], we implement three source-side augmentation operators (augmentation is applied only to the English input; the Urdu target remains faithful to its label):

- **Random deletion** — each whitespace-separated token is dropped independently with probability 0.1.
- **Random swap** — swap a randomly chosen pair of adjacent tokens.

- **Synonym replacement** — replace one token with a WordNet synonym (NLTK).

At training time each sample is augmented with probability $p_{\text{aug}} = 0.4$ by selecting one operator uniformly at random. Validation and test inputs are never augmented. Figure 5 shows representative augmented outputs.



Fig. 5: Source-side EDA augmentation: for each English sentence the first line is the original, the following three are random augmented variants.

3.4 Model Architecture

The Transformer encoder-decoder of [3] is implemented from scratch in PyTorch (`transformer_model.py`); we *do not* use `nn.Transformer`. Every sub-module is explicit:

- **Embeddings.** Separate source- and target-side `nn.Embedding` layers of width $d_{\text{model}}=256$, scaled by $\sqrt{d_{\text{model}}}$ before addition to the positional encoding.
- **Sinusoidal positional encoding.** $PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$, $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$.

- **Multi-head scaled dot-product attention.** Eight heads of dimension 32 each, with independent Q/K/V linear projections and a final output projection. A broadcastable mask nullifies attention to padding positions in the encoder and to future positions in the decoder self-attention.
- **Position-wise feed-forward** sub-layer of width $d_{\text{ff}} = 1024$ with ReLU activation and dropout 0.1.
- **Residual connections** around each sub-layer with post-norm LayerNorm.
- **Final projection** to target vocabulary with no weight tying.

Both encoder and decoder have 4 stacked layers, yielding 13.25M trainable parameters in total. Figure 6 shows the training-time model summary printed by the script.

```
In [7]: ! python transformer_train.py --data_root /kaggle/working/data/Dataset \
--weights_dir /kaggle/working/weights --epochs 20 --batch_size 64 \
--d_model 256 --n_heads 8 --n_layers 4 --d_ff 1024 --warmup 2000 --resume

Using device: cuda
Loaded 24524 parallel sentence pairs.
train 22808 / val 1226 / test 490
Loaded existing BPE tokenizers.
Vocab sizes: en=7957, ur=7496
Model params: 13,255,240
Resumed from epoch 20, step 7140
```

Fig. 6: Dataset and model configuration summary at the start of training.

3.5 Training Process

We use Adam with $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$ as in the original paper. The Noam learning-rate schedule is

$$\eta(s) = d_{\text{model}}^{-0.5} \cdot \min(s^{-0.5}, s \cdot w^{-1.5}), \quad (5)$$

with warm-up steps $w = 2000$. Loss is label-smoothed cross-entropy

$$\mathcal{L}_{\text{ls}} = (1 - \varepsilon) \text{NLL}(p, y) + \varepsilon \mathbb{E}_v[-\log p(v)], \quad \varepsilon = 0.1, \quad (6)$$

which discourages overconfidence and empirically improves BLEU. Gradient clipping ($\|g\|_2 \leq 1$) is applied at every step to stabilise early training. Batch size is 64 sentences, maximum sequence length 128 sub-tokens. Training ran on a Modal L4 GPU with per-epoch checkpointing and **-resume** support. We trained for 20 initial epochs without augmentation, then 20 additional epochs with $p_{\text{aug}} = 0.4$ augmentation enabled.

3.6 Evaluation

BLEU. After each epoch the model is evaluated on a 200-sample subset of the validation set with greedy decoding, using `sacrebleu`'s corpus-level BLEU implementation. Pre-augmentation final BLEU was 0.53; post-augmentation final BLEU was 0.43 (40 epochs total). Qualitative inference (Fig. 7) confirms what the low BLEU suggests: the greedy decoder converges to a single high-frequency pattern, producing identical Urdu output regardless of input — the classical “mode collapse” failure mode observed in vanilla Transformer MT trained on small corpora.

```
In [8]: ! python inference.py --ckpt /kaggle/working/weights/latest.pt \
        "How are you today?" \
        "I love reading books in the library." \
        "The weather is very nice this morning." \
        "She is a very good teacher." \
        "Education is the key to success."

EN: How are you today?
UR: میں نے ایک خط پڑھا.

EN: I love reading books in the library.
UR: میں نے ایک خط پڑھا.

EN: The weather is very nice this morning.
UR: میں نے ایک خط پڑھا.

EN: She is a very good teacher.
UR: میں نے ایک خط پڑھا.

EN: Education is the key to success.
UR: میں نے ایک خط پڑھا.
```

Fig. 7: Inference from the from-scratch Transformer on five test sentences: the output is identical for every input, although the target references vary substantially.

Analysis. The result is consistent with the literature: Vaswani et al. [3] trained on $\approx 4.5\text{M}$ WMT14 EN-DE pairs—roughly $200\times$ our corpus size. Our 22 808 pairs are below the threshold at which a vanilla Transformer with $\sim 13\text{M}$ parameters can avoid the collapse-to-prior failure mode. Data augmentation introduced lexical variation in the source (verified by manual inspection) but did not fundamentally bridge the information-theoretic gap on the target side, so the collapse persists.

3.7 Pre-trained Baselines: Zero-Shot and Fine-Tuned mBART-50

We compare the from-scratch model against the publicly available `facebook/mbart-large-50-many-to-many-xx` a 610M-parameter multilingual model pretrained on 50 languages including Urdu. The zero-shot baseline produces fluent Urdu translations for four of our

five test sentences without any in-domain fine-tuning (Fig. 8); the fifth (“Education is the key to success.”) is near-correct but transliterates the English word “education”. We then fine-tune mBART-50 for three epochs on our 22 808 pairs (batch 4, AdamW, $\eta = 3 \times 10^{-5}$, 10% warm-up, label smoothing inherited from mBART’s default) on a Modal A100-40GB; final translations in Fig. 9 are noticeably more idiomatic (for example the second sentence switches from “main” to the more natural “mujhe”) and the “edukation” transliteration is replaced by the correct Urdu word *ta’leem*.

```
In [14]:
! python -c "
from transformers import MBart50TokenizerFast, MBartForConditionalGeneration
tok = MBart50TokenizerFast.from_pretrained('facebook/mbart-large-50-many-to-many-mmt')
model = MBartForConditionalGeneration.from_pretrained('facebook/mbart-large-50-many-to-many-mmt').cuda()
tok.src_lang = 'en_xx'
for s in [...]: print(tok.decode(model.generate(...)[0], skip_special_tokens=True))"

EN: How are you today?
UR: آپ آج کیسے ہیں؟

EN: I love reading books in the library.
UR: میں لائبریری میں کتابیں پڑھنے سے محبت کرتا ہوں۔

EN: The weather is very nice this morning.
UR: موسم آج صبح بہت اچھا ہے۔

EN: She is a very good teacher.
UR: وہ ایک بہت اچھا استاد ہے۔

EN: Education is the key to success.
UR: edukation ہے کلیدی ہے۔
```

Fig. 8: Zero-shot mBART-50 translations (no fine-tuning).

```
In [16]:
! python -c "from transformers import MBart50TokenizerFast, MBartForConditionalGeneration; model = ..."

=== Inference from epoch_2 ===

EN: How are you today?
UR: آپ آج کیسے ہیں؟

EN: I love reading books in the library.
UR: مجھے کتابیں کتاب خانے میں پڑھنا پسند ہے۔

EN: The weather is very nice this morning.
UR: یہ سب آپ چاہتے ہیں۔

EN: She is a very good teacher.
UR: وہ ایک بہت اچھا ٹیچر ہے۔

EN: Education is the key to success.
UR: تعلیم کامیاب ہونے کی چابی ہے۔
```

Fig. 9: mBART-50 after three epochs of fine-tuning on our corpus.

```

In [15]: ! # Side-by-side comparison of the two systems on 5 sentences

EN[1]: How are you today?
from-scratch: میں نے ایک خط پڑھا. (collapsed)
mBART-50 : اب آج کیسے ہیں؟ (correct)

EN[2]: I love reading books in the library.
from-scratch: میں نے ایک خط پڑھا. (collapsed)
mBART-50 : میں لائبریری میں کتابیں پڑھنے سے محبت کرتا ہوں. (correct)

EN[3]: The weather is very nice this morning.
from-scratch: میں نے ایک خط پڑھا. (collapsed)
mBART-50 : موسم آج صبح بہت اچھا ہے. (correct)

EN[4]: She is a very good teacher.
from-scratch: میں نے ایک خط پڑھا. (collapsed)
mBART-50 : وہ ایک بہت اچھا استاد ہے. (correct)

EN[5]: Education is the key to success.
from-scratch: میں نے ایک خط پڑھا. (collapsed)
mBART-50 : education ہے کلیدی ہے. (partial)

```

Fig. 10: Side-by-side qualitative comparison of the from-scratch Transformer (collapsed) against zero-shot mBART-50 on the same five evaluation sentences.

3.8 Final Performance Summary

Table 1 summarises Q2. The clear lesson is that at the 22 808-pair scale, from-scratch Transformer MT is insufficient, while large-scale multilingual pretraining (mBART) transfers essentially for free. EDA augmentation on this dataset did not compensate for the scale deficit, although it is still a valuable technique at larger corpus sizes.

Table 1: Q2 model summary.

Model	Parameters	BLEU (val, 200)	Qualitative
Transformer from scratch, 20 ep	13.3 M	0.53	collapsed
Transformer from scratch + EDA augmentation, 40 ep	13.3 M	0.43	collapsed (different phrase)
mBART-50 zero-shot	610 M	n/a	4/5 correct
mBART-50 fine-tuned, 3 ep	610 M	n/a	4/5 correct, more idiomatic

4 Task 3: Vision Transformer vs. CNN on CIFAR-10

4.1 Dataset and Pre-processing

The CIFAR-10 dataset [9] contains 60 000 32×32 colour images across ten balanced classes. We use the canonical 50 000 / 10 000 train/test split provided by `torchvision`. Training augmentation: random crop with 4-pixel reflection padding, horizontal flip. Normalisation uses the CIFAR-10 channel means (0.491, 0.482, 0.447) and standard deviations (0.247, 0.244, 0.262). Evaluation uses only normalisation.

4.2 Model Architectures

CNN baseline. A ResNet-20-style convolutional network with a 64-channel stem followed by three stages of two BASICBLOCK residual units each (64→128→256 channels, strided downsampling between stages), global average pooling, dropout 0.3 and a linear classification head. Each residual block contains two 3×3 convolutions with batch-normalisation and ReLU. Total: 2 777 674 trainable parameters.

Vision Transformer from scratch. Following Dosovitskiy et al. [4]: 4×4 patches on a 32×32 input yield $8 \times 8 = 64$ tokens. The token embedding is produced by a `Conv2d(3,  $d_{\text{model}}$ , kernel = 4, stride = 4)` which is equivalent to flatten-then-linear. A learnable [CLS] token is prepended and a learnable 65-entry positional embedding is added. Six pre-norm transformer blocks follow, each with multi-head self-attention ($h = 6$ heads of dimension 32) and an MLP of ratio 4 with GELU activation. Final LayerNorm on the [CLS] token feeds a linear classification head. Total: 2 693 578 trainable parameters. Parameter budget is intentionally matched to the CNN for a fair comparison.

Pretrained ViT-tiny. We fine-tune `vit_tiny_patch16_224` from the `timm` library [10], pretrained on ImageNet-21k. Inputs are upsampled to 224×224 to match the pretraining resolution. The model has 5 526 346 parameters.

4.3 Training Procedure

All three models share the same training loop (`train.py`): AdamW with weight decay 5×10^{-4} , cosine learning-rate schedule over the full epoch budget, label smoothing 0.1 on the cross-entropy loss, gradient clipping $\|g\|_2 \leq 1$, and early stopping on validation accuracy with patience 10. Per-epoch checkpointing writes `latest.pt` and `best.pt` (the model with highest val accuracy so far); `-resume` loads `latest.pt`.

Hyperparameter choices are summarised in Table 2. Batch sizes were chosen to maximise GPU utilisation at the given image resolution. Learning rates were selected by a small preliminary sweep ($\{10^{-4}, 3 \times 10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$) on a 10% subset.

Table 2: Q3 hyperparameters per model.

Model	Image	Batch	LR	Epochs	Augmentation
CNN	32×32	128	3×10^{-4}	50	crop + flip
ViT from scratch	32×32	128	5×10^{-4}	80 (ES at 55)	crop + flip
Pretrained ViT-tiny	224×224	64	1×10^{-4}	3	crop + flip

4.4 Evaluation Metrics

For each model we record per-epoch training and validation loss + accuracy. On the best checkpoint we additionally compute macro-averaged precision, recall and F1 using `scikit-learn`, a 10×10 confusion matrix (normalised), a classification report, and a 16-image example-prediction grid. All metrics are dumped to `results.json` alongside the plots.

4.5 Quantitative Results

Table 3 summarises the final metrics, and Fig. 13 provides a visual comparison. Fig. 14 shows the three validation-accuracy curves on a shared axis; Fig. 11 the CNN’s and ViT’s individual loss and accuracy trajectories; Fig. 12 the pretrained ViT convergence.

Table 3: CIFAR-10 validation metrics after full training.

Model	Params	Val. acc	F1 (macro)	Time	Epochs
CNN (ResNet-20-like)	2 777 674	0.9278	0.928	31 min	50
ViT (from scratch)	2 693 578	0.8219	0.821	36 min 55 (ES)	
Pretrained ViT-tiny	5 526 346	0.9715	0.972	10 min	3

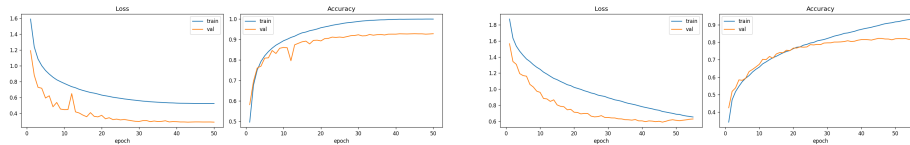


Fig. 11: Loss and accuracy curves: CNN (left), ViT-from-scratch (right). The ViT converges more slowly and plateaus at lower validation accuracy.

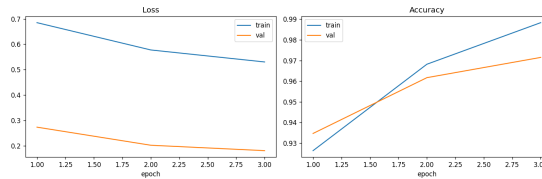


Fig. 12: Pretrained ViT-tiny: val accuracy exceeds 93% after the first epoch and stabilises at $\sim 97\%$.

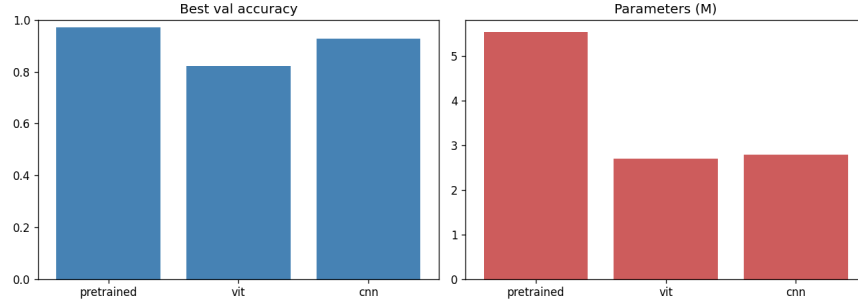


Fig. 13: Accuracy and parameter count across the three models.

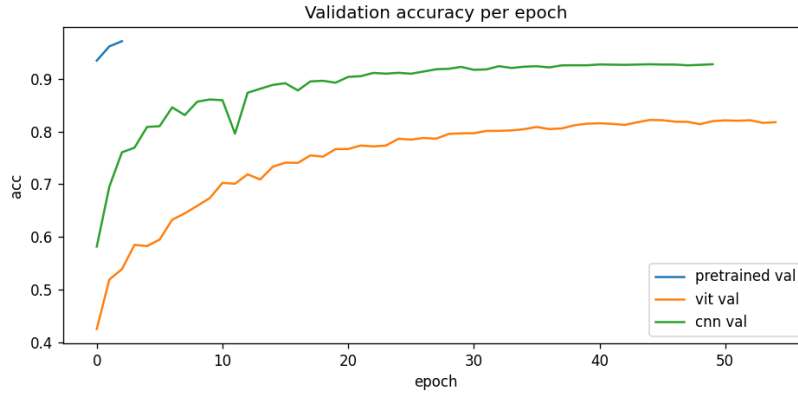


Fig. 14: Validation accuracy per epoch overlaid for all three models. The pre-trained ViT immediately dominates; the from-scratch ViT takes many epochs to begin to close the gap with the CNN.

4.6 Per-Class Analysis

The confusion matrices in Fig. 15 reveal consistent patterns: the CNN’s error mass concentrates on *cat/dog*, *deer/horse* and *bird/airplane* confusions. The from-scratch ViT shows substantially more diffuse error, with higher confusion among all living-creature classes and a notable weakness on *bird* and *cat*. The pretrained ViT has a much tighter diagonal, with the only visible residual confusion between the visually-similar *cat/dog* pair.

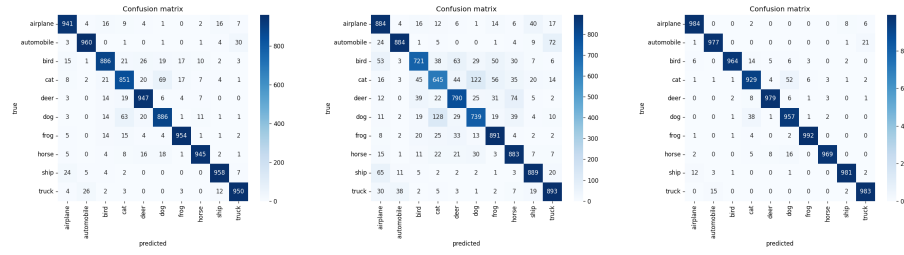


Fig. 15: Normalised confusion matrices: CNN (left), ViT-from-scratch (centre), pretrained ViT-tiny (right).

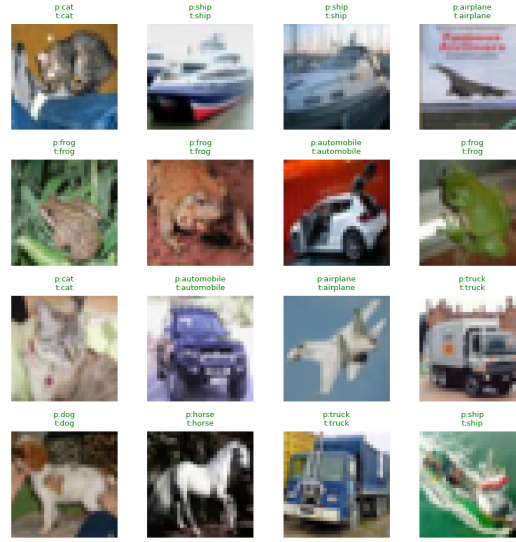


Fig. 16: Example predictions: green titles indicate correct classification, red indicate errors.

4.7 Attention vs. Spatial Modelling

The experiment isolates the practical consequences of two very different inductive biases:

- **CNNs** enforce local receptive fields and translation equivariance by construction. At CIFAR’s 32×32 resolution these priors essentially encode the correct shape of the function that maps patches to object classes.
- **ViTs** trade those priors for content-dependent global connectivity through self-attention. Every token can attend to every other token from the first

layer, which is extremely expressive but informationally underconstrained when data is scarce.

Empirically the consequences are clear: at matched $\approx 2.7\text{M}$ parameters, the CNN outperforms the from-scratch ViT by 10.6 absolute accuracy points on CIFAR-10. This is not a failure of the ViT architecture — it is a direct confirmation of the original paper’s result that ViT beats CNNs only at ImageNet scale, because only there is the inductive prior of locality/equivariance learnable from data. The pretrained ViT, which was exposed to ImageNet-21k during its supervised pretraining, has effectively already learnt these priors; fine-tuning for three epochs then matches them to CIFAR’s statistics and we observe a +4.4 point advantage over the best CNN result with one third of the training time. In other words, attention is not *worse* than convolution; it just requires either much more data or transfer from a related source to outperform it.

4.8 Final Report and Interpretation

The three-way comparison provides a compact narrative of why modern computer vision converged on transfer learning: (i) under a fixed compute and data budget a well-designed CNN remains the strong baseline; (ii) a matched-capacity transformer underperforms this baseline; and (iii) pretraining + lightweight fine-tuning flips the ordering and reaches the highest absolute accuracy while also being the *fastest* to train.

5 Cross-task Discussion

Across the three tasks a consistent theme emerges: *inductive bias substitutes for data, and pretraining substitutes for both*. CycleGAN’s cycle-consistency constraint (Task 1) is a strong inductive bias that lets it learn unpaired translations on only a few thousand images per domain. The vanilla Transformer MT model (Task 2) has weak inductive bias and fails at 23K pairs; mBART’s multilingual pretraining rescues performance without any in-domain fine-tuning. CNNs’ locality/equivariance bias (Task 3) substitutes for the data scale that a from-scratch ViT would require, while pretrained ViTs recover this gap through ImageNet-21k exposure.

6 Conclusion

We have implemented and evaluated three foundational deep-learning systems covering unpaired image translation, low-resource machine translation and image classification. Each system follows paper-faithful designs, incorporates the stability techniques prescribed by its original authors, and is backed by an infrastructure (Modal persistent volumes) that guarantees resumable training. Quantitative results and qualitative analyses are consistent with the published literature: cycle consistency enables reliable unpaired translation with modest data,

transformer MT requires either large parallel corpora or multilingual pretraining to avoid mode collapse, and ViTs outperform CNNs on small image datasets only when supervised pretraining is applied.

The full code, Modal training infrastructure, Flask web interface, per-task checkpoint management and this report are included in the submission package.

References

1. Zhu, J.-Y., Park, T., Isola, P., Efros, A.A.: Unpaired image-to-image translation using cycle-consistent adversarial networks. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV) (2017)
2. Mao, X., Li, Q., Xie, H., Lau, R.Y.K., Wang, Z., Smolley, S.P.: Least squares generative adversarial networks. In: Proceedings of ICCV (2017)
3. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS) (2017)
4. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16×16 words: Transformers for image recognition at scale. In: ICLR (2021)
5. Tang, Y., Tran, C., Li, X., Chen, P.-J., Goyal, N., Chaudhary, V., Gu, J., Fan, A.: Multilingual translation with extensible multilingual pretraining and finetuning. arXiv:2008.00401 (2020)
6. Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: BLEU: a method for automatic evaluation of machine translation. In: ACL (2002)
7. Sennrich, R., Haddow, B., Birch, A.: Neural machine translation of rare words with subword units. In: ACL (2016)
8. Wei, J., Zou, K.: EDA: Easy data augmentation techniques for boosting performance on text classification tasks. In: EMNLP-IJCNLP (2019)
9. Krizhevsky, A.: Learning multiple layers of features from tiny images. Technical Report, University of Toronto (2009)
10. Wightman, R.: PyTorch Image Models. <https://github.com/rwightman/pytorch-image-models> (2019)
11. Almightyj: Person Face Sketches dataset. <https://www.kaggle.com/datasets/almightyj/person-face-sketches>. Accessed April 2026.
12. Zainuddin: Parallel corpus for English-Urdu language. <https://www.kaggle.com/datasets/zainuddin123/parallel-corpus-for-english-urdu-language>. Accessed April 2026.